

Занятие 3. Что реально используют после 2023: SSD, delta-family, гибриды и sparse attention

Как сравнивать современные architectures, не превращая курс в каталог названий?

Автор: Андрей Ильинский, 2026

Четыре уровня сравнения

state equation

operator

evaluation algorithm

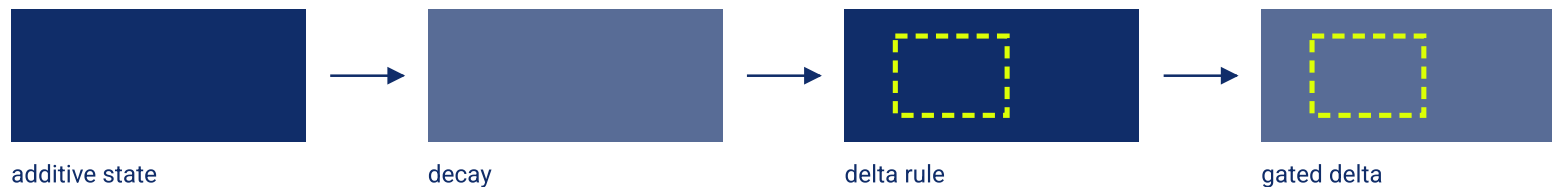
full architecture

Карточка стоит на одном и том же месте у каждого разбора.

Подсвечен уровень, на котором идёт сравнение прямо сейчас.

Остальные уровни остаются видимыми: совпадение на одном уровне не означает совпадения на других.

Что уже есть к началу занятия



Насколько выразительна transition и как вычислить sequence эффективно?

Generic finite-state recurrence

уровни сравнения

state equation

operator

evaluation algorithm

full architecture

$$S_t = S_{t-1} \mathcal{A}_t + U_t.$$

additive

$$\mathcal{A}_t = I$$

decay

$$\mathcal{A}_t = \gamma_t I$$

delta

$$\mathcal{A}_t = I - \beta_t k_t k_t^\top$$

Общая форма

Что общего у всех архитектур с конечным состоянием?

Unroll показывает, через что проходит старая запись

$$S_3 = \mathcal{A}_3 \mathcal{A}_2 U_1 + \mathcal{A}_3 U_2 + U_3.$$

$$S_t = \sum_{i=1}^t \mathcal{A}_t \mathcal{A}_{t-1} \cdots \mathcal{A}_{i+1} U_i.$$

Старый вход доходит до момента t только через упорядоченное произведение transitions.

Scalar transition делает survival factor явным

$$S_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t a_j \right) v_i k_i^\top.$$

content interaction

$v_i k_i^\top$ — что именно записано

temporal survival

state survival = $\prod_{j=i+1}^t a_j$

Из recurrence получается pairwise operator

$$\Omega_{ti} = \mathbf{1}[i \leq t] \left(\prod_{j=i+1}^t a_j \right) q_t^\top k_i.$$

$$y_t = \sum_{i=1}^T \Omega_{ti} v_i.$$

уровни сравнения

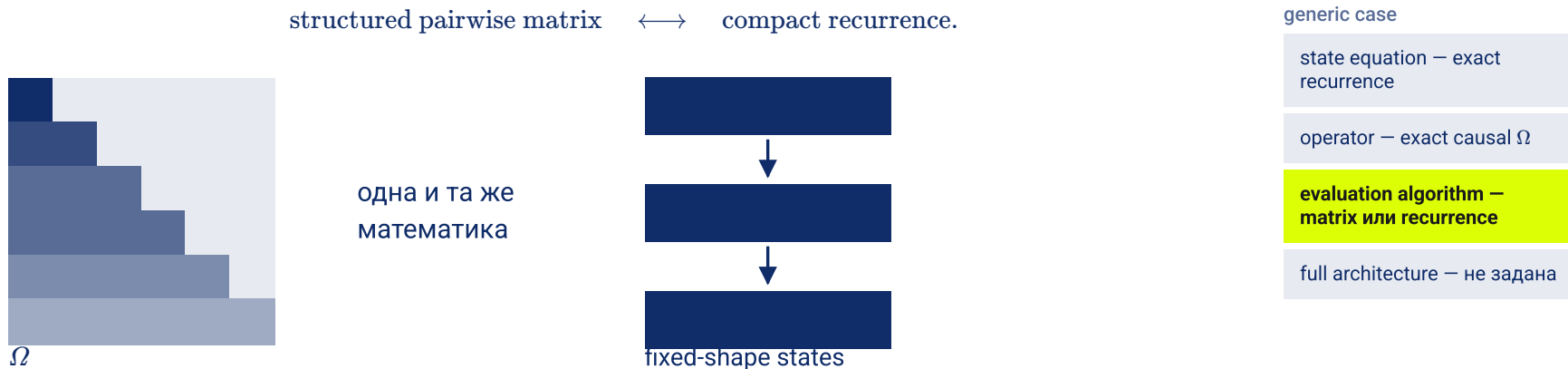
state equation

operator

evaluation algorithm

full architecture

Structured pairwise matrix $\leftrightarrow$ compact recurrence



SSD: finite state оставляет structure в большой matrix

Что можно сказать о cross-boundary interactions?

Разрезаем sequence после position m

$$\prod_{j=i+1}^t a_j = \left(\prod_{j=m+1}^t a_j \right) \left(\prod_{j=i+1}^m a_j \right).$$



уровни сравнения

state equation

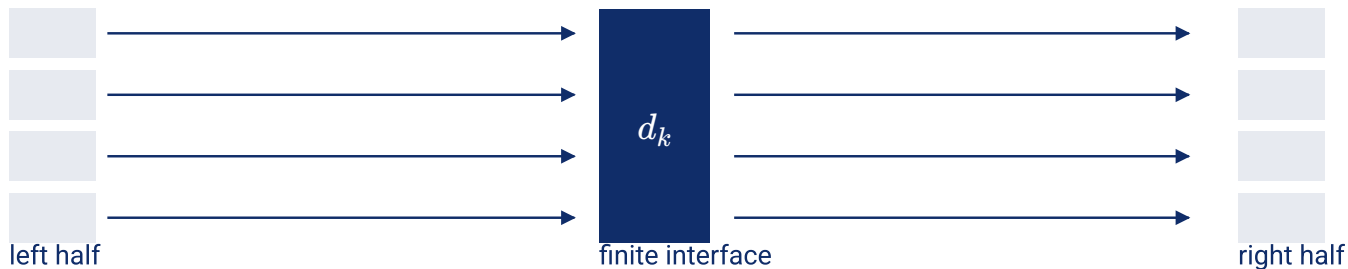
operator

evaluation algorithm

full architecture

Transition factors можно присоединить к query и key

$$q_t^{[m]} = \left(\prod_{j=m+1}^t a_j \right) q_t, \quad k_i^{[m]} = \left(\prod_{j=i+1}^m a_j \right) k_i.$$



уровни сравнения

state equation

operator

evaluation algorithm

full architecture

Cross-boundary block имеет bounded rank

уровни сравнения

state equation

operator

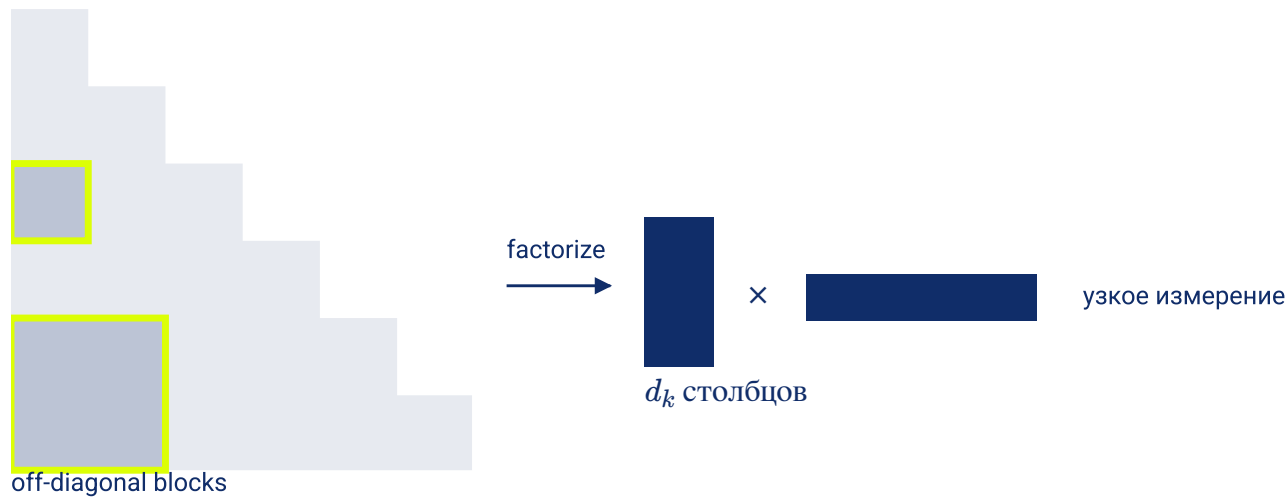
evaluation algorithm

full architecture

$$\Omega_{\text{right},\text{left}} = Q_{\text{right}}^{[m]} (K_{\text{left}}^{[m]})^\top.$$

$$\text{rank}(\Omega_{\text{right},\text{left}}) \leq d_k.$$

Semiseparable view



уровни сравнения

state equation

operator

evaluation algorithm

full architecture

Что утверждает SSD — и чего не утверждает

structured recurrence $\rightarrow$ structured causal matrix $\rightarrow$
alternative algorithms.

Эквивалентность взглядов на один operator — не утверждение о том, что архитектуры совпадают.

SSD / Mamba-2

state equation — structured SSM
class

operator — semiseparable
structured transformation

**evaluation algorithm — state-
space и structured matrix views**

full architecture — Mamba-2; не
identity с Transformer

Другая axis: насколько fine-grained управление finite state

scalar control $\rightarrow$ vector / channel-wise control.

Один scalar gate против channel-wise control

scalar gate

$$S = \begin{bmatrix} 10 & 1 \end{bmatrix}, \quad \alpha S = \begin{bmatrix} 5 & 0.5 \end{bmatrix}.$$

channel-wise gate

$$g = \begin{bmatrix} 0.1 \\ 0.9 \end{bmatrix}, \quad G = \text{diag}(g),$$

$$SG = \begin{bmatrix} 1 & 0.9 \end{bmatrix}.$$

уровни сравнения

state equation

operator

evaluation algorithm

full architecture

Toy abstraction смысла channel-wise forgetting, не paper-specific recurrence.

Delta-family: сравниваем design axis, а не названия

уровни сравнения

state equation

operator

evaluation algorithm

full architecture — зависит от family

fixed scalar forgetting

data-dependent scalar

global α_t + targeted β_t

vector / channel-wise control

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

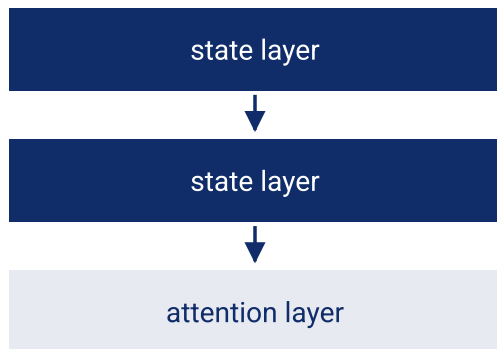
RWKV-7 и KDA располагаются на этой же оси, ближе к vector / channel-wise управлению.

Почему появляются гибриды

Что если fixed state хорош для части зависимостей, а explicit token retrieval всё ещё нужен?

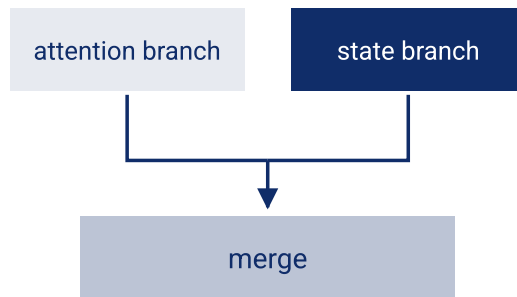
Два типа hybrid

layerwise



разные mixers на разных глубинах

parallel



две ветви внутри одного block

уровни сравнения

state equation

operator

evaluation algorithm

full architecture

Hybrid хранит два типа context representation

уровни сравнения

state equation

operator

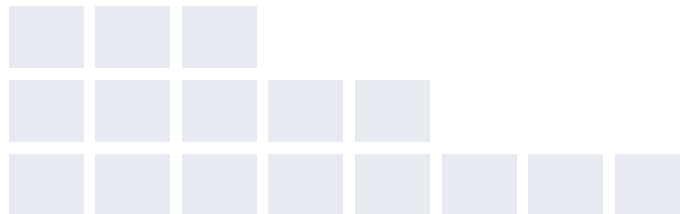
evaluation algorithm

full architecture

$$M_{\text{persistent}} = M_{\text{state}} + M_{\text{KV}}.$$



state layers: fixed



attention layers: token-indexed cache пастёт

Qwen3.5: 3 Gated DeltaNet + 1 attention



GDN · GDN · GDN · Attention

15 повторов → 60 layers

state layers

$$\frac{45}{60} = 0.75$$

attention layers

$$\frac{15}{60} = 0.25$$

75% layers ≠ 75% экономии KV-cache

уровни сравнения

state equation

operator

evaluation algorithm

full architecture

Карточка hybrid case

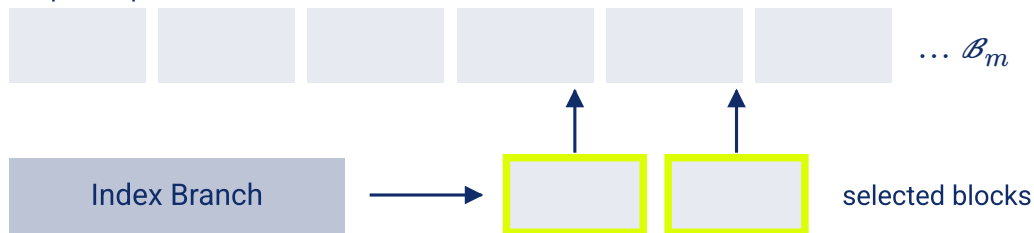
state equation	finite states в Gated DeltaNet layers + token-indexed cache в attention layers
operator	layerwise mixture двух mixers
evaluation algorithm	recurrent/state kernels + attention cache
full architecture	Qwen3.5; MoE sparsity — отдельная axis

Sparse attention меняет read, а не обязательно storage

Можно ли хранить много, но дорого читать только малую часть?

Learned sparse retrieval: храним pool, выбираем blocks

KV pool: хранится всё



$$\mathcal{I}_t = \text{TopK}_j s(\hat{q}_t, \mathcal{B}_j), \quad \mathcal{S}_t = \bigcup_{j \in \mathcal{I}_t} \mathcal{B}_j.$$

$$y_t = \text{Attention} \left(\hat{q}_t, \hat{K}_{\mathcal{S}_t}, V_{\mathcal{S}_t} \right).$$

уровни сравнения

state equation

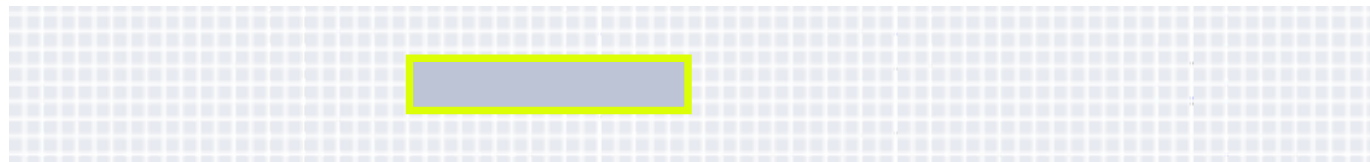
operator

evaluation algorithm

full architecture

Миллион stored tokens, 2048 tokens в main read

$$\frac{1\,000\,000}{64} \approx 15\,625.$$



15 625 blocks по 64 позиции

$32 \times 64 = 2048$ selected positions

store

1 000 000

main read

2 048

уровни сравнения

state equation

operator

evaluation algorithm

full architecture

Карточка sparse-attention case

state / cache	большой token-indexed KV pool
operator	attention только над selected set
evaluation algorithm	Index Branch → TopK blocks → exact attention на selected blocks
full architecture	MiniMax Sparse Attention как concrete example

Три разных ответа на проблему long context

MECHANISM	ЧТО ХРАНИТ	ЧТО ЧИТАЕТ QUERY	ГЛАВНЫЙ TRADE-OFF
Fixed-state recurrence	fixed matrix state	compressed state	finite-state bottleneck
Hybrid state + attention	state + часть token-indexed KV	зависит от layer	cache остаётся, coefficient меньше
Sparse attention	большой KV pool	selected blocks	storage растёт, main read сокращается

Индустрия выбрала не одну архитектуру, а набор механизмов. Гибрид — это распределение задач, а не признак неудачи линейных слоёв.

После занятия 3 меняется вопрос курса

уровни сравнения

state equation

operator

evaluation algorithm

full architecture

structured state

hybrid

sparse retrieval

Занятия 1–3: меняли ось A – context / backbone.

Занятие 4: фиксируем A и меняем ось B – сам способ генерации.